

Iniziato	Friday, 21 June 2024, 14:20
Stato	Completato
Terminato	Friday, 21 June 2024, 15:19
Tempo impiegato	59 min. 38 secondi
Valutazione	11,5 su un massimo di 12,0 (96%)

Si illustri il ciclo di vita di un oggetto esemplificandone i passi utilizzando uno specifico linguaggio OO.

Consideriamo il linguaggio Java su JVM.

Un oggetto ha un ciclo di vita ben specificato. All'inizio dell'esecuzione del programma, la JVM alloca un'area di memoria continua, detta managed heap, dove poi conservare gli oggetti allocati a runtime. Questa memoria in realtà può crescere o diminuire di spazio dinamicamente (in base ai parametri dell'invocazione della JVM), ma per questo esempio consideriamo sia costante.

Quando viene richiesta la creazione di un'istanza di una classe, tramite keyword new nel linguaggio Java, la JVM esegue questi passi:

1. Calcola la dimensione in memoria che l'oggetto occuperebbe, analizzando tutti i campi della classe.
2. Controlla che nel managed heap ci sia abbastanza spazio per contenere tutto l'oggetto. In caso contrario, prova ad invocare il Garbage Collector per liberare memoria e se anche dopo questo non c'è abbastanza memoria solleva un'eccezione per mancanza di memoria (OutOfMemoryException)
3. Crea un puntatore alla posizione attuale del "NextObjPtr" (ovvero un puntatore al primo indirizzo libero del managed heap) poi sposta il puntatore della dimensione dell'oggetto calcolata al punto 1.
4. Invoca il costruttore della classe specificata passando il puntatore creato al punto 3 come 'this'. Il costruttore procede all'inizializzazione della classe.
5. Restituisce al chiamante della new il riferimento all'area di memoria creato al punto 3, che adesso dovrebbe (se il costruttore ha fatto un buon lavoro) rappresentare un oggetto ben inizializzato.

Dopo questi passaggi l'oggetto viene utilizzato dal chiamante, e rimane in memoria finché necessario.

Quando l'oggetto non è più "necessario" (in base alla strategia di liberazione della memoria del Garbage Collector) la JVM procede a:

1. Chiamare il metodo finalize() dell'oggetto, una volta disponibile anche al programmatore utente ma adesso deprecato.
2. Marcare lo spazio di memoria una volta preso dall'oggetto come di nuovo disponibile, così da permetterne l'utilizzo da parte di altri oggetti.

Le strategie per marcare un oggetto come "non necessario" sono molte. Ne analizziamo due:

1. **Reference counting**, utilizzato per esempio dal sistema COM, in cui a fianco dell'oggetto viene conservato il numero di puntatori attualmente attivi verso l'oggetto. Ogni volta che viene creato un puntatore all'oggetto il numero viene incrementato e ogni volta che viene rimosso un puntatore il numero viene decrementato. Una volta che raggiunge 0, l'oggetto viene finalizzato e la memoria liberata. Problemi di questo approccio sono

- la non resistenza ai **riferimenti circolari**, in quanto in quel caso entrambi gli oggetti avranno sempre un refcount > 0 e quindi non verranno mai finalizzati (e la rispettiva memoria liberata)

- il problema della **deallocazione in cascata**, dove ogni singola deallocazione potrebbe scatenare una cascata di refcount che passano da 1 a 0 e quindi forzare il runtime a procedere sequenzialmente nella finalizzazione e nella liberazione della memoria. Ad esempio, supponiamo di avere oggetti A, B, C, D, con -> che indica una dipendenza data da un campo, se esiste solamente A in "scope" e A -> B -> C -> D, l'uscita dallo scope di A creerebbe una cascata di deallocazioni, in quanto il runtime deallocherebbe prima A, si accorgerebbe che B ha refcount = 0 e solo allora deallocherebbe B e così via

2. **Tracing**, di cui una modalità è il cd. Mark and Sweep (ci sono altre modalità, ad esempio le Generazioni utilizzate dai più recenti runtime JVM, ma il principio è lo stesso)

Il processo è composto da due fasi:

- durante la prima, fase di "marcatura", tutti gli oggetti "radice", ovvero quelli che hanno riferimenti sullo stack attuale, quelli che hanno riferimenti nei registri CPU o che hanno riferimenti globali, vengono marcati come in uso. Si procede quindi alla marcatura ricorsiva di tutti i riferimenti che fanno parte degli oggetti radice e di quelli già marcati, saltando quelli già marcati precedentemente, finché non si finisce.

- nella seconda fase, di "sweep", gli oggetti presenti nel managed heap che non sono stati marcati durante la prima fase vengono finalizzati e ne viene liberata la memoria

Anche le strategie di Tracing soffrono di diversi problemi, come per esempio:

- il fatto che il momento in cui avviene un ciclo di garbage collection non sia noto al programmatore, in quanto questo dipende da molte variabili, molte di cui esterne al controllo del programmatore. Questo è critico nei sistemi che

devono avere latenza molto bassa, in quanto un ciclo di GC può rallentare di non poco l'applicazione

- il cosiddetto momento di Stop The World, che in realtà non è esclusivo ai GC Tracing ma in generale è un problema della garbage collection, ovvero il momento in cui, dopo aver indicato come libero lo spazio di memoria occupato da oggetti garbage collected, la memoria occupata deve essere compattata, e quindi devono essere aggiornati TUTTI i riferimenti presenti in memoria. Durante questa fase tutto deve essere fermo, in quanto qualsiasi scrittura / lettura dalla memoria potrebbe corromperla poichè non in uno stato consistente.

- il fatto che le operazioni compiute durante il tracing utilizzano risorse, quindi creando overhead per il programma (problema anche per il ref counting, ma lì so esattamente quando può verificarsi)

- il fatto che, in Java ad esempio, non ci sia un modo esplicito da parte del programmatore di liberare un oggetto, costringendo quindi il GC ad occuparsi anche degli oggetti creati temporaneamente e di cui il programmatore può garantire il non utilizzo dopo un determinato punto.

Si illustri (con almeno un esempio) il Pattern Decorator.

Il pattern Decorator è una valida alternativa all'ereditarietà nel caso ci siano un quantitativo di combinazioni di classi la cui implementazione sarebbe lunga e ripetitiva. In particolare può essere utilizzato per "aggiungere" delle funzionalità a delle classi, in particolare un albero di classi, in modo tale da poter utilizzare ogni decoratore su qualsiasi altro elemento e rendere facile la combinazione di decoratori diversi.

Supponiamo per esempio di avere un videogioco, dove ogni animale è rappresentato da una classe. Supponiamo di avere però variazioni dell'animale, in quanto a, per esempio, abilità. Supponiamo che ogni animale abbia il metodo `attacca(Persona)`

Abbiamo quindi 3 animali:

- Gatto: il metodo `attacca` morde la persona
- Cane: il metodo `attacca` graffia la persona
- Maiale: il metodo `attacca` carica la persona

E tre abilità:

- Combattente
- Difensore
- Mago

Senza utilizzare il pattern decorator, dovrei creare le combinazioni di tutte le classi, ad esempio `GattoCombattente`, `CaneMago` e così via. Con il pattern decorator, posso invece creare una classe "Combattente" da poter utilizzare per decorare tutte e tre le classi animali.

Deve esistere una superclasse, classe astratta o interfaccia, `Animale`, che specifica la presenza di un metodo `"attacca(Persona)"`. Ogni animale deve estendere (o implementare) la classe / interfaccia.

Deve esistere inoltre, una classe astratta `Decorator` che implementa `Animale`, da cui poi deriveranno tutte le abilità. La classe potrebbe anche prendersi parte del lavoro dei decoratori e salvare in una variabile da esporre ad ogni sottoclasse un puntatore all'oggetto decorato

```
abstract class Decorator extends Animale {  
    private Animale decorated; // da aggiungere un getter protected  
    abstract void attacca(Persona)
```

```
    public Decorator(decorated: Animale) {  
        this.decorated = decorated;  
    }  
}
```

Adesso potrei creare la classe `Mago` che estende `Decorator` e potrei specificare che nel metodo `attacca`, prima (o anche dopo) di invocare il rispettivo metodo della classe decorata, faccia anche un danno magico alla `Persona` che sta attaccando.

```
...  
public void attacca(Persona p) {  
    getDecorated().attacca(p);  
    p.damage(10, DamageType.Magic)  
}  
...
```

Potrei quindi fare la stessa cosa con tutte le altre classi `Decorator`.

La gerarchia sarebbe quindi:

```
- Animale
- Gatto
- Cane
- ...
- Decorator
- Mago
- Combattente
- ...
```

Potrei anche creare un caneMagoCombattente, creando un Combattente(Mago(cane)), e funzionerebbe comunque tutto.

Domanda **3**

Completo

Punteggio ottenuto 4,0 su 4,0

Si illustri il modello LMU nei VCS indicandone vantaggi e svantaggi.

Il modello Lock-Modify-Unlock è uno dei modelli per la gestione del lavoro concorrente (inteso come lavoro di più persone che modificano i file) nei sistemi di controllo versione. Nel modello Lock-Modify-Unlock ogni file, prima di essere modificato, deve essere "bloccato" (lock), chiedendo al server VCS il permesso assoluto e completo di fare modifiche al file. Se il file non è attualmente "bloccato", il server risponderà in maniera affermativa, altrimenti restituirà un errore e questo permetterà di capire che qualcun altro ci sta già lavorando sopra. Dopo il "blocco" del file, colui che l'ha bloccato può apportare le proprie modifiche, con la garanzia che non verranno sovrascritte da modifiche concorrenti. Alla fine delle modifiche, il file viene eventualmente aggiornato (se ci sono state modifiche) e "sbloccato" dall'utente, e questo permette agli altri utenti di poter bloccare nuovamente il file e apportare le proprie modifiche.

Il modello LMU ha molti svantaggi, ma anche qualche vantaggio. Gli svantaggi sono:

- la mancanza di possibilità da parte di più utenti di modificare il file in punti diversi in contemporanea, in quando il file è bloccato per intero da un singolo utente
- l'obbligo di dover essere online per tutti gli utilizzatori del sistema VCS, in quanto è necessaria una comunicazione con il server per bloccare e sbloccare i file
- la falsa sicurezza data dal blocco del file, in quanto, soprattutto per file di codice, la modifica di file che dipendono l'uno dall'altro ha conseguenze anche distruttive, e questo non viene neanche considerato dal modello LMU
- il non sblocco accidentale di un file (es utente che dimentica di sbloccarlo dopo aver finito il lavoro) rallenta di molto il lavoro, in quanto si deve chiedere all'amministratore del VCS di sbloccarlo manualmente

E' quindi in generale da preferirsi un altro modello, quello del Update-Modify-Merge, in cui le modifiche fatte da più utenti sullo stesso file vengono "unite" e questo permette di lavorare in contemporanea sullo stesso file.

Nonostante ciò, il modello LMU eccelle per i file binari, in quanto l'intero file è (normalmente) oggetto di modifica, e il merge basato sulle righe (che si fa normalmente per l'altro modello) è inapplicabile.